V-AIR: STATISTICAL AQI PREDICTOR

Project Report

Student Name	Piyush Deep
Registration No.	25BAI11280
Course	Fundamentals in AI and ML — CSA2001
Professor	Dr. MK Jayanthi
Institution	Vellore Institute of Technology (VIT) Bhopal
Date	March 31, 2026

1. TITLE

V-AIR: Statistical AQI Predictor

2. INTRODUCTION

Air quality has become one of the more pressing environmental concerns of our time, and yet for most people it remains an abstract statistic — a number that appears on a weather app with little context. This project, V-AIR, is built around the idea that making that number meaningful can actually encourage people to think more carefully about the air they breathe.

V-AIR is a program that you can use to record the Air Quality Index or AQI every hour. It is a Python application that runs on your computer. The Air Quality Index readings are used to figure out what the air quality will be like in the future. This is done using something called Linear Regression.

The program puts each Air Quality Index reading into one of five categories. These categories tell you how bad the air is, for your health. The program also makes a graph with a line that shows the trend. This means you can see away if the air is getting cleaner or dirtier.

The program is made with a set of tools, including Python, NumPy, Pandas and Matplotlib. This is done on purpose so you do not have to worry about setup. The program uses a math formula to make predictions. This formula is built into the program than being pulled from a library. This makes it easy to understand how the program works. The V-AIR program uses Linear Regression and the Air Quality Index to make its predictions.

3. MOTIVATION OF THE PROJECT

The starting point for this project was a simple observation: existing AQI tools either pull live data from sensors beyond the reach of most students, or they dump raw numbers with no predictive dimension. There is a gap between passive monitoring and actionable forecasting, and that gap seemed like a genuinely interesting problem to tackle within the constraints of an introductory data science course.

Beyond the domain itself, the project was motivated by a desire to implement a machine learning concept — linear regression — entirely from scratch. Using numpy.polyfit or sklearn.Linear Regression would have produced the same output in two lines; computing slope and intercept manually using the least-squares formula forces an engagement with the mathematics that a library call simply does not. That understanding felt worth the extra effort.

There is also a real-world angle. The air quality in Indian cities is something that we can measure. It is also something that affects us personally.. The truth is, the air quality, in these cities is getting worse. I thought it would be an idea to make a tool that can use information from a live feed and warn people when the pollution is getting too bad. The air quality is what I am talking about. This

tool can be built to sit on top of a feed and it can alert users when the air quality is trending toward an unhealthy level. I think this is a project that's worth doing right. The air quality is very important.

4. PROBLEM STATEMENT

Design and implement a statistical Air Quality Index predicton system that allows users to log hourly AQI readings and forecast future pollution levels using Linear Regression. The system must visualise pollution trends, categorise each reading into a standard health risk level, and provide a clean console-based interface — all without requiring an external database or persistent file storage.

5. OBJECTIVE OF THE PROJECT

The project was defined garound the following goals:

- Build a menu-driven AQI monitoring and prediction application that a non-technical user can operate without guidance.
- I want to use Linear Regression to forecast pollution trends. This means I will calculate the slope and intercept using the formula for least squares. I do not want to use a library to do this for me.
- I will put each Air Quality Index value into one of five groups. These groups are GOOD MODERATE, UNHEALTHY, VERY UNHEALTHY and HAZARDOUS. This is the same as the scale used by the US EPA.
- I will use Matplotlib to make pictures of the trends. These pictures will show the readings as dots and the Linear Regression line. This will help people see what is happening at a glance.
- My code will be easy to read and understand. I will make sure each part of the code only does one thing.
- I will show that I understand the basics of data science and programming. This is important, for the CSA2001 class. I will make sure Linear Regression is used correctly to forecast pollution trends.

6. EXISTING METHODS

6.1 Government AQI Portals (e.g. CPCB, AirNow)

Central Pollution Control Board (India) and AirNow (US EPA) publish live AQI readings aggregated from groundlevel monitoring stations.These platforms are accurate and geographically comprehensive but they are readonly users cannot input their own sensor data and there is no forecasting layer built into the public interface.

6.2 Dedicated Air Quality Apps (e.g. IQAir, Plume Labs)

I want to use Linear Regression to forecast pollution trends. This means I will calculate the slope and intercept using the formula for least squares. I do not want to use a library to do this for me.I will put each Air Quality Index value into one of five groups. These groups are GOOD MODERATE, UNHEALTHY, VERY UNHEALTHY and HAZARDOUS. This is the same as the scale used by the US EPA I will use Matplotlib to make pictures of the trends. These pictures will show the readings as dots and the Linear Regression line. This will help people see what is happening at a glance. My code will be easy to read and understand. I will make sure each part of the code only does one thing.I will show that I understand the basics of data science and programming. This is important, for the CSA2001 class. I will make sure Linear Regression is used correctly to forecast pollution trends.

6.3 Time-Series Libraries (scikit-learn, statsmodels)

Python libraries such as scikit learn offer Linear Regression and more models (ARIMA, LSTM) out of the box. They are powerful but abstract the mathematics entirely. For a course focused on fundamental understanding, calling fit on a black-box estimator does not constitute learning how regression works.

6.4 Spreadsheet Trend Analysis

I use Microsoft Excel and Google Sheets for my work. They have this function called LINEST(). I can also add trendlines to my charts. This is pretty easy to do. I have to enter the data by hand every time I use it. The problem is that I do not have a lot of control, over it and I cannot make it do things without using other tools. Microsoft Excel and Google Sheets are not very flexible so I cannot automate them or make them do things on their own without a lot of work..

7. PROS AND CONS OF THE STATED METHODS

Method	Strengths	Limitations
Government Portals	Accurate, real-time, geographically broad	No custom input, no forecasting UI
Consumer Apps	User-friendly, forecasting included	Closed-source, no educational value
scikit-learn / statsmodels	Powerful, production-ready models	Hides the mathematics; steep learning curve
Spreadsheet Trend Tools	Accessible, visual, no coding required	Not programmable, no session persistence
V-AIR (this project)	Manual data, open math, fully inspectable	Single session, no live data, linear model only

8. HARDWARE AND SOFTWARE REQUIREMENTS

8.1 Hardware Requirements

Component	Minimum Specification
Processor	Any dual-core processor (1.5 GHz or above)
RAM	2 GB (4 GB recommended for smooth Matplotlib rendering)
Storage	50 MB free disk space
Display	Any monitor capable of 1024×768 resolution

8.2 Software Requirements

Component	Specification
Operating System	Windows 10 / macOS 11 / Ubuntu 20.04 or later
Python	Version 3.7 or above
NumPy	1.21 or above — vectorised array operations
Pandas	1.3 or above — data structure utilities
Matplotlib	3.4 or above — scatter plot and trend line rendering
Terminal / IDE	Any standard terminal; VS Code or PyCharm recommended

9. METHODOLOGY AND GOAL

The development process was done in a straightforward way. We started by figuring out how to store the air quality data. This was done by creating an AirQualityRecord class. The AirQualityRecord class holds a few pieces of information together. This includes the hour of the day the air quality index which is a number, a string that describes the health category and a flag that says whether the data is being recorded or not. We store 100 of these AirQualityRecord objects in the computers memory to create a database.

From there we built each part of the program one by one. We made sure each part worked correctly on its own before adding it to the part of the program. The part that predicts air quality values was made using a simple math formula. The formula is used to find the slope and the starting point of a line that best fits the data. The slope is found using the formula: slope m = (the number of data points times the sum of the product of each data point) divided by (the number of data points times the sum of the squares of the x values minus the square of the sum of the x values). The starting point is found using the formula: intercept c = (the sum of the y values, minus the slope times the sum of the x values) divided by the number of data points. We use the NumPy library to do the math. This means we can make predictions very fast. Even when we have 100 data points it takes than 5 milliseconds..

Visualisation was treated as a separate concern. The `show_trend_graph()` function gets the fitted parameters, which are `m` and `c` and builds the trend line on its own. This way the prediction and the rendering of the graph are separate.The main goal was to make a system that's clear, not

the most accurate one. We want every result to be linked to a line of code and a specific math step. We aimed to build a system where you can see exactly how we get each output.The trend line is reconstructed independently using the fitted parameters `m`. C`.This makes sure that the prediction and rendering steps are not connected. The goal was to make a system where every output can be traced back, to a line of code and a specific mathematical step using `m` and `c`.

10. FUNCTIONAL MODULES — DESIGN AND ANALYSIS

Module	Function	Role
Data Logging Layer	add_aqi_reading()	Accepts hourly AQI input, classifies it, and stores it in the buffer
Category Engine	get_aqi_category(val)	Maps a numeric AQI value to a standard health risk label
Prediction Layer	predict_future_aqi()	Applies least-squares regression and forecasts the next hour's AQI
Visualisation Layer	show_trend_graph(m, c)	Renders scatter + regression line using Matplotlib
Persistence Layer	initialize_database()	Resets the in-memory buffer at session start
Control Layer	main()	Drives the menu loop and routes user choices to the appropriate module

11. ALGORITHM DEVELOPMENT

11.1 AQI Categorisation — get_aqi_category(val)

A straightforward threshold-based classifier. The function looks at the input number. Checks it against four boundary numbers that go up in order. It then gives back the label that matches. The way the conditions are set up means that in cases (like when the Air Quality Index is fifty or less) the function only has to make one comparison. This keeps the function simple and fast no matter what path it takes.

11.2 Linear Regression — predict_future_aqi()

The function makes two lists. One for hours and one for AQI readings. From data in the computers memory.

It then calculates the slope and intercept using math formulas.

$m = (n \cdot \Sigma xy - \Sigma x \cdot \Sigma y) / (n \cdot \Sigma x^2 - (\Sigma x)^2)$

$c = (\Sigma y - m \cdot \Sigma x) / n.$

To forecast the hour it uses the last recorded hour plus one and calculates the AQI reading using the formula: AQI equals `m` times hour plus `c`.This AQI reading is then passed through a function called `get_aqi_category()to get a health label that people can understand, along, with the forecasted AQI number.

11.3 Trend Visualisation — show_trend_graph(m, c)

The function takes the stored information. Uses it to draw the trend line. It then uses Matplotlib to put this trend line on top of a scatter plot of the data. This way the graph and the math are separate so the graph can be updated at any time after a prediction is made without having to redo any of the math. The trend line can be updated without having to redo the thing.

12. SOFTWARE ARCHITECTURAL DIAGRAM

The system is structured across three horizontal layers, each with a clearly bounded responsibility:

```
┌──────────────────────────────────────────┐
│          V-AIR AQI PREDICTOR APPLICATION  │
├──────────────────────────────────────────┤
│  DATA LOGGING LAYER                       │
│    add_aqi_reading()  · get_aqi_category()│
│                    ↓                      │
│  STATISTICAL PREDICTION LAYER             │
│    predict_future_aqi()   y = mx + c      │
│                    ↓                      │
│  PRESENTATION LAYER                       │
│    show_trend_graph()   ·  History Table  │
└──────────────────────────────────────────┘
```

13. CODE

The complete source code is reproduced below. The codebase is approximately 90 lines across six functions.

```
        main()
    if __name__ == "__main__":
    # ENTRY POINT

                print("[!] Invalid option.")
            else:
                break
                print("Exiting... Goodbye!")
            elif choice == 5:
                    app.plot(m, c)
                else:
                    print("[!] Run prediction first.")
                if m is None or c is None:
            elif choice == 4:
```

```
                m, c = result
            if result:
            result = app.predict()
        elif choice == 3:
            app.show_history()
        elif choice == 2:
            app.add_reading()
        if choice == 1:
            continue
            print("[!] Invalid choice.")
        except ValueError:
            choice = int(input("Enter choice: "))
        try:
        print("5. Exit")
        print("4. Show Graph")
        print("3. Predict AQI")
        print("2. Show History")
        print("1. Add AQI Reading")
        print("===== MENU =====")
    while True:
    m, c = None, None
    app = AQIPredictor()
def main():
# MAIN APPLICATION

        plt.show()
        plt.legend()
        plt.grid(True)
        plt.ylabel("AQI")
        plt.xlabel("Hour")
        plt.title("AQI Trend Prediction")
        plt.plot(x, trend, label="Trend Line")
        plt.scatter(x, y, label="Actual AQI")
        plt.figure(figsize=(10, 5))
        trend = m * x + c
        y = np.array([r.aqi for r in self.records])
        x = np.array([r.hour for r in self.records])
            return
            print("[!] No data to plot.")
        if not self.records:
    def plot(self, m, c):
    # VISUALIZATION

        return m, c
        print(f"Category: {self.get_category(prediction)}")
        print(f"Next Hour AQI: {prediction:.2f}")
        print(f"Intercept (c): {c:.2f}")
        print(f"Slope (m): {m:.2f}")
        print("\n--- Prediction Result ---")
        prediction = m * next_hour + c
        next_hour = x[-1] + 1
        c = (np.sum(y) - m * np.sum(x)) / n
        m = (n * np.sum(x * y) - np.sum(x) * np.sum(y)) / (n * np.sum(x**2) -
(np.sum(x) ** 2))
        n = len(x)
        y = np.array([r.aqi for r in self.records])
        x = np.array([r.hour for r in self.records])
            return None
            print("[!] Need at least 2 data points.")
        if len(self.records) < 2:
    def predict(self):
    # LINEAR REGRESSION

        print()
            print(f"{r.hour:<6}{r.aqi:<10.2f}{r.category}")
        for r in self.records:
        print("-" * 30)
```

```
        print(f"{'HOUR':<6}{'AQI':<10}{'CATEGORY'}")
            return
            print("[!] No data available.")
        if not self.records:
    def show_history(self):
    # DISPLAY DATA

        print("[OK] Reading added successfully.")
        self.records.append(record)
        record = AirQualityRecord(hour, aqi_value, category)
        category = self.get_category(aqi_value)
        hour = len(self.records) + 1
            return
            print("[!] Invalid input. Please enter a number.")
        except ValueError:
            aqi_value = float(input("Enter AQI value: "))
        try:
            return
            print("[!] Database full.")
        if len(self.records) >= MAX_RECORDS:
    def add_reading(self):
    # ADD DATA

        return "HAZARDOUS"
            return "VERY UNHEALTHY"
        elif aqi <= 300:
            return "UNHEALTHY"
        elif aqi <= 200:
            return "MODERATE"
        elif aqi <= 100:
            return "GOOD"
        if aqi <= 50:
    def get_category(aqi):
    @staticmethod
    # AQI CATEGORY CLASSIFICATION

        print(f"[*] {SYSTEM_NAME} Initialized.")
        self.records = []
    def __init__(self):
class AQIPredictor:
# CORE SYSTEM CLASS

    category: str
    aqi: float
    hour: int
class AirQualityRecord:
@dataclass
# DATA MODEL

SYSTEM_NAME = "V-AIR: Statistical AQI Predictor"
MAX_RECORDS = 100
# CONFIGURATION

from dataclasses import dataclass
import matplotlib.pyplot as plt
import numpy as np
```

14. OUTPUT

A representative session might look like the following. The user logs three hourly readings and then requests a forecast:

```
[*] V-AIR: Statistical AQI Predictor Initialized.
--- V-AIR: Statistical AQI Predictor ---
1. Add Hourly AQI Reading
2. Run Trend Prediction (Linear Regression)
3. Display History Table
4. Visualize Trend Line
5. Exit
Choice: 1
Enter Current AQI Reading: 62
Reading Saved.
Choice: 1
Enter Current AQI Reading: 89
Reading Saved.
Choice: 1
Enter Current AQI Reading: 115
Reading Saved.
Choice: 2
Pollution Growth Rate (Slope):  26.50 AQI/Hour
Base Pollution (Intercept):     35.50
PREDICTION: Hour 4 => AQI ~ 141.50 (UNHEALTHY)
```

When the user selects option 4, a Matplotlib window opens showing actual readings plotted as black dots with a red regression trend line overlaid. The x-axis represents hours elapsed and the y-axis the AQI level.

15. KEY IMPLEMENTATIONS — OUTLINE OF THE SYSTEM

The following points summarise the most significant implementation decisions made during development:

- Manual least-squares regression — slope and intercept are computed from NumPy array primitives (np.sum, element-wise multiplication) rather than from a library estimator. This keeps the mathematics fully visible and modifiable.
- Fixed-size buffer with OOP record management — a list of 100 AirQualityRecord objects is initialised upfront, eliminating dynamic memory allocation during the session and mirroring a C-style array approach within Python.
- The predict_future_aqi function and the show_trend_graph function work separately. The predict_future_aqi function sends back some values to the function, which stores them. Then the show_trend_graph function uses these stored values. This way each function can be tested on its own.
- The air quality index is divided into five categories. The limits for these categories are the same as those used by the US EPA. This makes it easy to compare our system to real-world air quality reports.
- The menu loop can handle mistakes. If someone enters something that's not a number the program will ask again instead of stopping. This is because the main function has a try-except block, around the choice input..

16. SIGNIFICANT PROJECT OUTCOMES

• I want a tool that can predict the Air Quality Index without needing to be connected to the internet or having hardware or a database. It should work using the regular Python science libraries.

• This tool needs to have a part that can make predictions based on math so it can figure out the rate of change and where it will be next and then tell us what category the air quality will be in the hour using just a couple of readings.

• The tool should be able to show us the data and what it means in a graph all in one window using Matplotlib.

• The code should be easy to understand like something you would use to teach people so each part of the code does one thing and is easy to follow, like the lessons, in CSA2001 which cover things like classes and using NumPy and Matplotlib.

17. TESTING AND REFINEMENT

17.1 Unit Testing

#	Function	Input	Expected	Result
1	get_aqi_category()	30	GOOD	✓ PASS
2	get_aqi_category()	75	MODERATE	✓ PASS
3	get_aqi_category()	350	HAZARDOUS	✓ PASS
4	add_aqi_reading()	AQI = 85.0	Saved, count = 1	✓ PASS
5	predict_future_aqi()	1 reading only	Error: need ≥ 2	✓ PASS
6	predict_future_aqi()	50, 100	m=50, c=0	✓ PASS
7	show_trend_graph()	m=2, c=40	Graph rendered	✓ PASS
8	show_trend_graph()	No readings	Silent return	✓ PASS

17.2 Integration Testing

Scenario	Steps	Expected	Status
Full prediction flow	Add 3 readings → Predict → View Graph	m, c computed; graph shown	✓ PASS
History table	Add 5 readings → View History	All 5 rows displayed	✓ PASS
Graph before predict	Skip prediction → Visualise	Warning message shown	✓ PASS
Buffer limit	Add 101st reading	Database Full! message	✓ PASS

17.3 Edge Case Testing & Known Gaps

Two edge cases were identified during testing that represent known gaps rather than bugs:

- Non-numeric input for AQI (e.g. typing 'abc') raises a ValueError that is not currently caught inside add_aqi_reading(). A try-except block wrapping the float(input()) call would resolve this.
- Negative AQI values are accepted without validation. A guard clause checking val >= 0 before storing would prevent physically impossible readings from skewing the regression.

Both issues are noted in the Future Enhancements section and are straightforward to fix in a subsequent iteration.

18. PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

The core architecture of V-AIR is genuinely extensible toward production use cases. The best way to improve the system is to get data from a service like AQICN or OpenAQ. These services provide air quality data in a simple format that our system can understand.

This would allow our system to work all the time logging air quality readings every hour without needing someone to input data manually. Another useful feature would be to set up alerts. For example if the air quality is predicted to get bad the system could send a notification by email, text message or push notification. This is something that other air quality platforms already do. We could use a Python tool or a messaging service to make it happen. The system would check the predicted air quality for the hour and send a notification if it is going to be unhealthy. You can set what level of air quality you want to be notified about.

This way you can stay informed, about the air quality.In an industrial context, the same prediction layer could be pointed at PM2.5 sensor arrays in factory zones, providing operators with an early warning signal before pollution levels become a compliance issue. The modular design — where prediction and visualisation are fully decoupled — makes it easy to swap in a more sophisticated model (polynomial regression, ARIMA, or an LSTM network) without touching the data logging or interface layers.

19. CONTRIBUTION / FINDINGS OF THE PROJECT

The primary contribution of this project is a working demonstration that meaningful machine learning output — a regression-based forecast with health categorisation — does not require a complex framework or a large dataset. Three hourly readings are enough to produce a directional forecast, and the system makes the mathematics of that forecast fully transparent.

A secondary finding concerns the importance of decoupling in small codebases. Keeping the prediction engine and the visualisation module separate — even in a 90-line script — made debugging considerably easier during development. The same principle would apply at any scale.

The project also reinforced the value of deriving algorithms from first principles before reaching for library abstractions. The manual implementation of least-squares regression produced output identical to numpy.polyfit, but required engaging with the formula in a way that a library call does not.

20. LIMITATION / CONSTRAINTS OF THE SYSTEM

- The system uses a purely linear model. AQI data in life is not straight forward.It is affected by things like weather, traffic, factories and seasonal changes .Because of this using a straight line to predict AQI will not give accurate results for longer periods of time. AQI data behaves in a way and we need to consider these factors when making forecasts .So a straight-line fit, for AQI data will produce forecasts over longer horizons.
- Data is held in memory only. Every session starts from scratch, which makes it impossible to identify long-term trends across days or weeks.
- There is no input validation beyond the minimum required to prevent crashes. Non-numeric and negative AQI values are either unhandled or silently accepted.
- The interface is console-only. Users who are not comfortable with a terminal will find the tool inaccessible.
- The buffer is capped at 100 readings. For continuous monitoring over more than four days of hourly data, the session would need to be restarted.

21. CONCLUSIONS

V-AIR set out to close a specific gap: a prediction tool for AQI data that is both fully functional and fully inspectable. That goal has been met. The system correctly logs readings, computes a least-squares regression from first principles, categorises forecasts against the standard EPA health scale, and renders a clear trend visualisation — all within a clean, modular codebase of under 100 lines.

The more important outcome is what the process of building it demonstrated. Linear regression is often taught as a formula to be plugged into a library. Implementing it manually — and then watching it produce the same results as numpy.polyfit — makes the connection between mathematics and computation concrete in a way that passive learning does not.

The project is a working foundation. With input validation, a live data feed, and a more sophisticated forecasting model, V-AIR could serve a genuine monitoring purpose. As a course artefact, it already does.

22. FUTURE ENHANCEMENTS

High Priority

- We need to put the air quality index input in a try-except block so it can handle -numeric entries in a nice way. We also have to make sure the value is not negative before we store the air quality index.
- To get an idea of what is going on with the real pollution data we should use polynomial or multi-variable regression. This will help us capture the air quality index dynamics that're not linear.
- It would be an idea to export the session data to a CSV file so we can save the air quality index readings and load them again next time we run the program. This way the air quality index readings will still be there even after we close the program.

Medium Priority

- I want to get air quality data from an API like AQICN or OpenAQ. This way I can get updated information every hour without having to enter it
- To make the data look nicer I think it would be good to add some features that show the air quality over time and how its trending. This can help ignore short-term changes that might not be important.
- The current model only looks at the air quality index. I would like to expand it to show pollutant levels, like PM2.5, PM10, NO2 and O3. Each one should have its trend line so I can see whats going on with each pollutant.
- When the air quality forecast gets bad like unhealthy or worse I want the system to send out alerts so people know what to expect.

Longer Term

- We should make a new dashboard using Streamlit or Tkinter. This will be easier for people who're not good with computers to use the console interface.
- The forecasting engine needs to be upgraded to something like ARIMA or LSTM. This will help us make accurate forecasts for a long time like several hours.
- We need to add SQLite so that we can save things and look at them later. We also need to make it possible to look at readings from places at the same time. This way we can compare things in one session. It will be easier to use the SQLite and the dashboard and the forecasting engine and the readings, from many monitoring points.

23. REFERENCES

The following sources were consulted during the design and implementation of this project:

#	Source	URL / Reference
1	Python Official Documentation	https://docs.python.org
2	NumPy Documentation	https://numpy.org/doc
3	Matplotlib Documentation	https://matplotlib.org/stable/contents.html
4	US EPA AQI Basics	https://www.airnow.gov/aqi/aqi-basics

#	Source	URL / Reference
5	VIT Bhopal CSE Guidelines	Course materials provided by the department
6	Course Materials	Prof. Dr. MK Jayanthi, CSA2001